Aim : **To illustrate the use of Binary Semaphore and Counting Semaphore for Task Synchronization using FreeRTOS**.

Theory :

FreeRTOS

FreeRTOS is a lightweight, open-source Real-Time Operating System (RTOS) made for use in embedded systems.It helps manage multiple tasks, schedules them properly, allows tasks to work together smoothly, handles memory efficiently, and lets tasks communicate with each other.

One of the most important tools for making tasks work together in FreeRTOS is called a semaphore.

Semaphores are used to control access to shared resources and help tasks coordinate with each other.

FreeRTOS has two main types of semaphores:

1.Binary Semaphore

A binary semaphore can be in one of two states:

- Available (1)

- Unavailable (0)

It is mainly used for:

- Synchronizing tasks

- Signaling events

- Communicating between an interrupt and a task

Unlike a mutex, a binary semaphore does not keep track of which task owns it.

One task (or an interrupt) gives the semaphore, and another task takes it.

Applications of binary semaphores include:

- Detecting a button press

- Signaling from a sensor interrupt

- Synchronizing two tasks

Algorithm :

```
#include <Arduino_FreeRTOS.h>
#include <semphr.h>

SemaphoreHandle_t BinarySem;
SemaphoreHandle_t CountSem;

//---------------- Binary Semaphore ----------------//

void BinaryGiveTask(void *pvParameters)
{
    while (1)
    {
        Serial.println("Task1: Giving Binary Semaphore");
        xSemaphoreGive(BinarySem);

        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void BinaryTakeTask(void *pvParameters)
{
    while (1)
    {
        if (xSemaphoreTake(BinarySem, portMAX_DELAY) == pdTRUE)
        {
            Serial.println("Task2: Binary Semaphore Received");
        }
    }
}
void CountGiveTask(void *pvParameters)
{
    while (1)
    {
        xSemaphoreGive(CountSem);

        Serial.println("Task3: Counting Semaphore Given");

        vTaskDelay(500 / portTICK_PERIOD_MS);
    }
```

```
}

void CountTakeTask(void *pvParameters)
{
    while (1)
    {
        if (xSemaphoreTake(CountSem, portMAX_DELAY) == pdTRUE)
        {
            Serial.println("Task4: Counting Semaphore Taken");
        }

        vTaskDelay(1500 / portTICK_PERIOD_MS);
    }
}

void setup()
{
    Serial.begin(9600);

    BinarySem = xSemaphoreCreateBinary();

    CountSem = xSemaphoreCreateCounting(5, 0);

    xTaskCreate(BinaryGiveTask,
                "Task1",
                128,
                NULL,
                1,
                NULL);

    xTaskCreate(BinaryTakeTask,
                "Task2",
                128,
                NULL,
                1,
                NULL);

    xTaskCreate(CountGiveTask,
                "Task3",
                128,
```

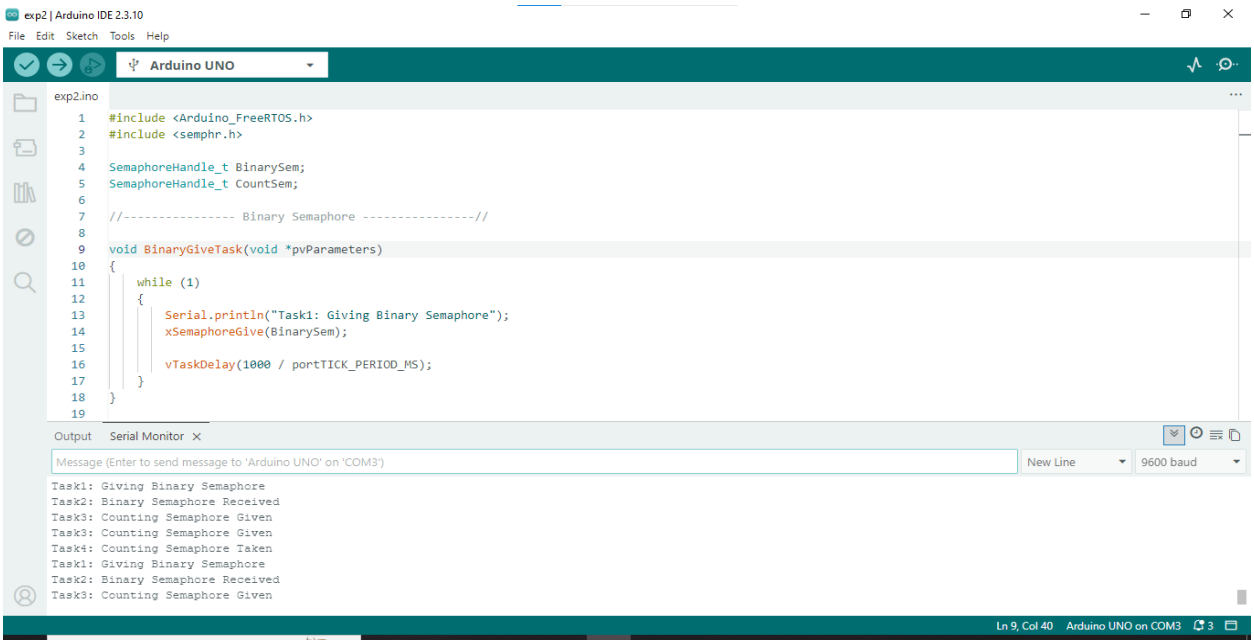
```
                NULL,
                1,
                NULL);

    xTaskCreate(CountTakeTask,
                "Task4",
                128,
                NULL,
                1,
                NULL);
    vTaskStartScheduler();
}
void loop()
{
}
```

Result : The Binary Semaphore and Counting Semaphore were **successfully created using FreeRTOS**.The Binary **Semaphore** was **used** to **coordinate** between two **tasks**, and the Counting **Semaphore** was **used** to **manage multiple semaphore signals**.The **tasks ran based** on when the **semaphores** were **available**, **showing correct synchronization** between them.

Conclusion : The **experiment helped** us **see** how **semaphores** are **used** to **synchronize tasks** in **FreeRTOS**.The Binary Semaphore was **good** for **sending** a **simple signal** between two **tasks**, while the Counting Semaphore could **hold multiple signals** and **let** them be **taken** one at a **time**.**So**, both **kinds** of **semaphores work well** for **coordinating tasks** and **controlling resources** in **real-time embedded systems**.

Output :


exp2 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino UNO
exp2.ino
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
SemaphoreHandle_t BinarySem;
SemaphoreHandle_t CountSem;
//---------- Binary Semaphore ----------//
void BinaryGiveTask(void *pvParameters)
{
while (1)
{
Serial.println("Task1: Giving Binary Semaphore");
xSemaphoreGive(BinarySem);
vTaskDelay(1000 / portTICK_PERIOD_MS);
}
}
Output Serial Monitor
Message (Enter to send message to 'Arduino UNO' on 'COM3')
New Line
9600 baud
Task1: Giving Binary Semaphore
Task2: Binary Semaphore Received
Task3: Counting Semaphore Given
Task3: Counting Semaphore Given
Task4: Counting Semaphore Taken
Task1: Giving Binary Semaphore
Task2: Binary Semaphore Received
Task3: Counting Semaphore Given
Ln 9, Col 40
Arduino UNO on COM3
3